Instituto Tecnológico de Costa Rica
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación
Curso: IC-6600 Principios de Sistemas Operativos
Profesor: M.Sc. Esteban Arias Méndez

Proyecto Programado

Entregable 1: Documento de Diseño del Sistema DSM

Implementación de un Sistema de Memoria Compartida Distribuida (DSM) con Paginación bajo Demanda y Coherencia MSI

Estudiantes:
Sergio Andrés Zúñiga Castillo, carné 2024146884
José Ignacio Madriz Ortiz, carné 2024089190

Periodo: II Semestre 2026
Fecha de entrega: Noviembre 6, 2026

Abstract

This document presents the design of a small Distributed Shared Memory (DSM) system for Linux, written in C++ with a C-compatible public interface. Every node is one run of the same program and all nodes are peers connected in a full horizontal TCP mesh. A shared configuration file sets the page size, the pages each node contributes, the hosts and the port range. Pages are globally numbered and each page has a fixed home node derived from its number. Access to a page without permission raises SIGSEGV, the handler sends a request and waits on a semaphore, and the network thread installs the page and opens its permission with mprotect. Coherence follows an MSI protocol with only the three states Modified, Shared and Invalid. The document specifies the general architecture, the configuration data, the data structures with their sizes, the binary message protocol with its exact field layout, and the initial API draft.

Índice

1. Introducción 3
2. Arquitectura general 3
    2.1 Nodos, tabla de nodos y heartbeat 3
    2.2 Hilos de un nodo 3
    2.3 Espacio de direcciones y nodo home 4
    2.4 Coherencia MSI 5
    2.5 Fallo de página y sincronización 5
3. Datos de configuración 6
4. Estructuras y tipos de datos 6
5. Protocolo de mensajes 8
    5.1 Encabezado 8
    5.2 Mensajes 8
    5.3 Atención de una solicitud en el home 9
6. Borrador de la API 10
7. Conclusiones 11
Bibliografía 11

1. Introducción

El proyecto programado del curso consiste en construir un sistema de Memoria Compartida Distribuida (DSM, *Distributed Shared Memory*) en espacio de usuario sobre Linux: varias computadoras sin memoria física común ofrecen a una aplicación un único espacio de direcciones, y las páginas viajan por la red cuando un nodo las necesita [1]. La idea sigue la memoria virtual compartida de Li y Hudak [2], que usa el mecanismo de paginación bajo demanda del sistema operativo [3].

Este documento cubre en detalle la arquitectura general, el diseño temprano de la API, el protocolo de mensajes y los datos de configuración. El sistema se escribe en C++ para Linux con `g++` y `-pthread`, la API pública se declara `extern "C"`. Los tamaños de `sem_t` y de los punteros corresponden a Linux x86-64 con glibc.

2. Arquitectura general

2.1 Nodos, tabla de nodos y heartbeat

Un **nodo** es una ejecución del programa `dsmnode`, en una terminal de una máquina o en otra computadora. Todos los nodos son iguales: no hay maestro ni roles de cliente y servidor, y cada nodo se conecta por TCP con todos los demás. Cada nodo guarda una **tabla de nodos** con la dirección IP, el rango de páginas que aporta y su estado (en línea o caído) para todos los demás.

- **Unión (JOIN).** Al arrancar, el nodo lee el archivo de configuración, abre sus puertos y envía `JOIN` a cada host de la lista. Cada nodo que lo recibe registra al nuevo nodo y responde `JOIN_ACK` con sus propios datos. Los hosts que no responden aún no están en ejecución y se registran cuando ellos mismos envíen su `JOIN`. La unión de un nodo solo agrega un rango de páginas nuevo al sistema.
- **Heartbeat.** Cada `heartbeat_ms` el nodo envía `HEARTBEAT` (una señal de vida) a todos los demás. Si pasan `timeout_ms` sin recibir nada de un nodo, se marca como caído, y vuelve a estar en línea cuando llega otro mensaje suyo.
- **Canales.** Cada nodo escucha en tres puertos: datos, control y heartbeat (sección 3). Para cada par de nodos, cada uno abre sus propias conexiones y solo envía por ellas. Lo que recibe llega por las conexiones que aceptó.

2.2 Hilos de un nodo

Las aplicaciones corren dentro del proceso `dsmnode`, porque `SIGSEGV` y `mprotect` solo actúan sobre el espacio de direcciones del propio proceso [4]. La consola de `dsmnode` las inicia con el comando `run`. Un nodo tiene estos hilos:

Hilo	Función
Hilos de usuario	Ejecutan la aplicación, usan la API y generan los fallos de página.

Hilo	Función
Hilo de red (*Network Listener Thread*)	Un solo hilo con `poll` sobre los tres puertos de escucha y todas las conexiones aceptadas. Atiende los mensajes sin bloquearse nunca.
Hilos de servicio	Hilos cortos que el hilo de red crea para `PAGE_REQUEST` y `FREE` cuando el nodo es el home de la página, porque esas operaciones esperan respuestas de otros nodos.
Hilo de heartbeat	Envía `HEARTBEAT` y marca como caídos a los nodos sin actividad.
Hilo principal	Consola: `run`, `nodes`, `stats`, `pages`.

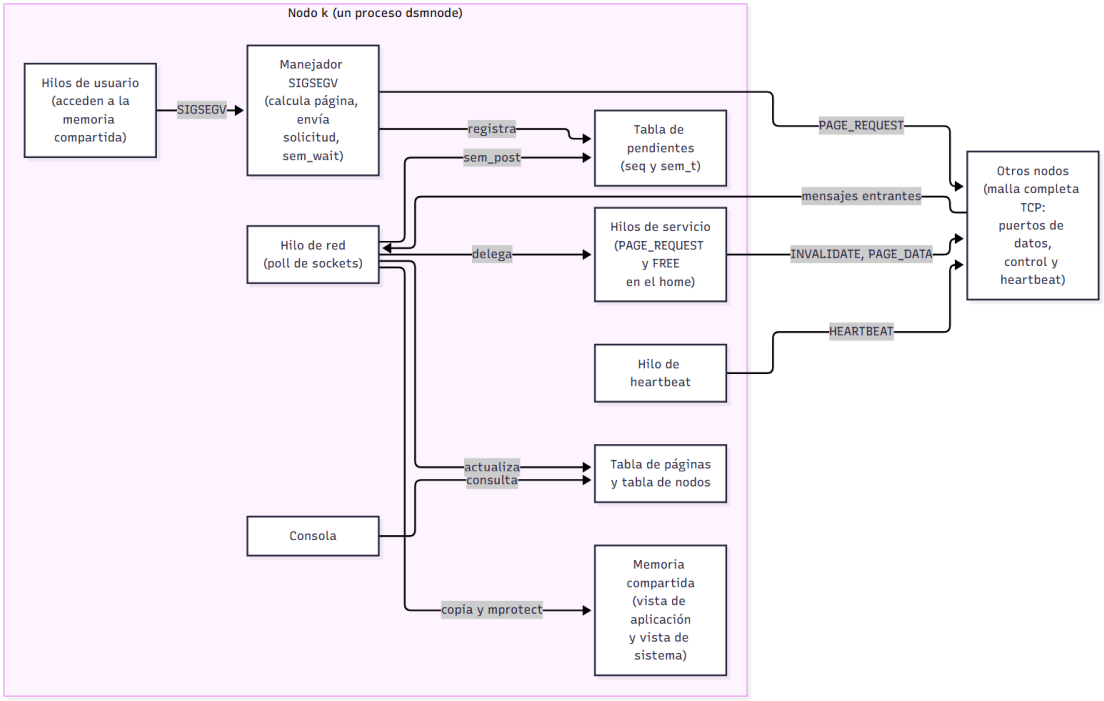


Figura 1. Arquitectura de un nodo dsmnode y su comunicación con los demás nodos

2.3 Espacio de direcciones y nodo home

Las páginas tienen una numeración global única. Con `N` hosts en la configuración, el sistema tiene `N * pages_per_node` páginas. El **nodo home** de una página se calcula directamente de su número:

```
home(page) = page / pages_per_node
```

El identificador `node_id` de un nodo es su posición en la lista `host` de la configuración, así que el nodo `k` es el home de las páginas `k * pages_per_node` hasta `(k + 1) * pages_per_node - 1`. Es el algoritmo de administrador distribuido fijo de Li y Hudak [2]: no hay un protocolo para averiguar quién administra una página, y cada nodo administra la memoria que aporta [1]. Si un nodo no está en línea, sus páginas no se pueden asignar ni acceder.

Todos los nodos reservan el mismo rango de direcciones virtuales, que empieza en `dsm_base` (parámetro de configuración), de modo que la dirección de la página `p` es `dsm_base + p * page_size` en todos los nodos y los punteros dentro de la memoria compartida valen en cualquier nodo. La dirección se reserva con `mmap` y sin permisos (`PROT_NONE`), solo ocupa memoria física lo que el nodo realmente tiene. La memoria se crea con `memfd_create` y se proyecta **dos veces** con `MAP_SHARED`: la *vista de aplicación* en `dsm_base`, con los permisos que fija `mprotect` página por página, y la *vista de sistema*, siempre de lectura y escritura, que el hilo de red usa para copiar páginas sin abrir el permiso a los hilos de usuario [4].

El home de una página guarda su entrada de directorio: el **dueño** (el nodo con la versión más reciente) y el **conjunto de copias** (los nodos con una copia válida, el dueño incluido). Además registra el proceso al que pertenece la página y su orden dentro de la asignación, que permiten responder las consultas del enunciado. Esa información vive en la **tabla de páginas** del nodo (sección 4), que tiene una entrada por cada página del sistema; en las páginas que el nodo no administra solo el campo de estado es válido.

2.4 Coherencia MSI

Cada nodo mantiene para cada página uno de tres estadosa. No se agrega ningún estado más: como todo cambio de estado de una página nace de un mensaje de su home y el home atiende una solicitud por página a la vez, no hay estados transitorios.

Evento	Estado antes	Estado después	Mensajes
El nodo lee una página que no tiene	I	S	`PAGE_REQUEST`(S) al home y `PAGE_DATA`(S) de vuelta. El dueño anterior pasa de M a S.
El nodo escribe una página que no puede escribir	S	M	`PAGE_REQUEST`(M) al home, `INVALIDATE` a las demás copias y `PAGE_DATA`(M) de vuelta. Una página en I pasa primero a S y luego a M.
Otro nodo escribe la página	S o M	I	`INVALIDATE` al nodo, o `PAGE_REQUEST`(M) si es el dueño.

Una página recién asignada está en M en su home, llena de ceros. Una página liberada o nunca asignada está en I en todos los nodos, y acceder a ella es un error.

2.5 Fallo de página y sincronización

El manejador de `SIGSEGV` se instala con `sigaction` y `SA_SIGINFO` [4]. Solo calcula la página con `si_addr`, decide el permiso que falta según el estado de la página (en I pide lectura, en S pide escritura), envía la solicitud al home, y llama a `sem_wait` sobre el semáforo de una entrada de la **tabla de pendientes** identificada por el número de

secuencia de la solicitud. El hilo de red recibe `PAGE_DATA`, copia los datos con la vista de sistema, abre el permiso con `mprotect`, actualiza el estado y llama a `sem_post`; el hilo de usuario sale del manejador y repite la instrucción. Si hay un error o el home está caído, el hilo recibe el estado de error por la misma entrada, escribe un mensaje y termina el proceso: esta versión no tolera la caída de nodos.

Las tablas se protegen con dos clases de mutex, ninguna dentro del manejador. `table_mutex` protege los campos de `PageEntry` y de `NodeInfo` en secciones críticas cortas. `home_lock[64]` es un arreglo de mutex que serializa en el home las solicitudes sobre una misma página (índice `page % 64`): un hilo de servicio lo mantiene durante toda la operación. No hay interbloqueo porque quien espera respuestas es siempre un hilo de servicio o de usuario, y el hilo de red, que las entrega, nunca espera.

Una asignación cabe en el rango de un solo home, así que mide como máximo `pages_per_node` páginas. El sistema admite hasta 32 nodos, solo direcciones IPv4 y no replica datos para tolerar fallas.

3. Datos de configuración

Todos los nodos leen el mismo archivo `dsm.conf`, de líneas `parametro = valor`; las líneas `host` se repiten una vez por nodo y su orden fija el `node_id`. Los parámetros son:

Parámetro	Tipo	Valor por defecto	Significado
`page_size`	`uint32`	4096	Tamaño de página en bytes. Múltiplo de 4096 para poder usar `mprotect`.
`pages_per_node`	`uint32`	256	Páginas que aporta cada nodo (256 páginas de 4 KB son 1 MB).
`heartbeat_ms`	`uint32`	1000	Período del `HEARTBEAT`.
`timeout_ms`	`uint32`	5000	Silencio tras el cual un nodo se marca caído.
`port_base`	`uint16`	50000	Primer puerto del rango de la pareja (ejemplo, ver abajo).
`dsm_base`	`uint64`	`0x100000000000`	Dirección virtual donde empieza la memoria compartida.
`host`	texto	(obligatorio)	Dirección IPv4 o nombre del nodo; entre 1 y 32 líneas.

Ejemplo de `dsm.conf` para dos nodos:

```
page_size      = 4096
pages_per_node = 256
heartbeat_ms   = 1000
timeout_ms     = 5000
port_base      = 50000
```

```
dsm_base       = 0x100000000000
host = 192.168.1.10
host = 192.168.1.11
```

Cada nodo usa tres puertos, uno por canal. El puerto se calcula con `puerto = port_base + 3 * node_id + rol`, con el rol 0 para datos, 1 para control y 2 para heartbeat. Con 32 nodos se usan 96 puertos.

Rango de puertos. De momento se usa el rango de 50000 a 50099 (`port_base = 50000`). Ese rango cambiará cuando el profesor asigne el definitivo (solo cambia el valor de `port_base`).

Al recibir un `JOIN`, el nodo valida que el `node_id`, la IP y el `num_pages` coincidan con su propia copia de la configuración (`host` y `pages_per_node`). Si no coinciden, el `JOIN_ACK` responde con error y la carga vacía.

4. Estructuras y tipos de datos

Las tipos son de ancho fijo y sin relleno implícito. El archivo `dsm_types.h` se compila como C++ y cada estructura lleva su `static_assert` con el tamaño.

Constantes: `MAX_NODES = 32`, `MAX_PENDING = 256`, `HOME_LOCKS = 64`, `QUERY_MAX = 256` (entradas por consulta), `NODE_NONE = 0xFF` y `PAGE_NONE = 0xFFFFFFFF`.

Tipo	Uso	Tamaño
`MsgHeader`	Encabezado de todos los mensajes	16 B
`NodeInfo`	Entrada de la tabla de nodos y carga de `JOIN`	16 B
`PageEntry`	Entrada de la tabla de páginas y carga de la consulta de páginas	16 B
`NodeStats`	Estadísticas de un nodo	32 B
`AllocReq`	Carga de `ALLOC_REQUEST`	8 B
`QueryReq`	Carga de `QUERY_REQUEST`	12 B
`Pending`	Entrada de la tabla de pendientes (solo en memoria)	64 B
`Config`	Configuración leída del archivo (solo en memoria)	2080 B

```c
#include <stdint.h>
#include <assert.h>          /* static_assert en C11; palabra reservada en C++ */
#include <semaphore.h>       /* sem_t, solo para Pending */

typedef struct MsgHeader {   /* ver tabla de la sección 5.1 */
    uint8_t  type, sender, status, flags;
    uint32_t seq, page, payload_len;
} MsgHeader;
static_assert(sizeof(MsgHeader) == 16, "MsgHeader");

typedef struct NodeInfo {
    uint32_t ipv4;           /* orden de red */
    uint32_t first_page;     /* node_id * pages_per_node */
    uint32_t num_pages;      /* pages_per_node */
```

```
    uint8_t  node_id;
    uint8_t  state;          /* 0 = caído, 1 = en línea */
    uint16_t reserved;
} NodeInfo;
static_assert(sizeof(NodeInfo) == 16, "NodeInfo");

typedef struct PageEntry {
    uint32_t proc_id;        /* (node_id << 16) | proc; 0 = libre */
    uint32_t copyset;        /* bit i = el nodo i tiene copia válida */
    uint32_t order;          /* índice de la página dentro de su asignación */
    uint8_t  state;          /* estado MSI local: 0 = I, 1 = S, 2 = M */
    uint8_t  owner;          /* nodo con la versión más reciente */
    uint16_t reserved;
} PageEntry;
static_assert(sizeof(PageEntry) == 16, "PageEntry");

typedef struct NodeStats {
    uint32_t total_pages, free_pages;      /* del rango propio */
    uint32_t pages_m, pages_s, pages_i;    /* páginas del sistema por estado
local */
    uint32_t faults, pages_sent, pages_recv;
} NodeStats;
static_assert(sizeof(NodeStats) == 32, "NodeStats");

typedef struct AllocReq { uint32_t npages, proc_id; } AllocReq;
static_assert(sizeof(AllocReq) == 8, "AllocReq");
typedef struct QueryReq { uint32_t kind, first_page, count; } QueryReq;
static_assert(sizeof(QueryReq) == 12, "QueryReq");

typedef struct Pending {     /* solo C++, x86-64 con glibc */
    sem_t    sem;            /* 32 B; el solicitante espera aquí */
    void    *out;            /* destino de la respuesta (consultas) */
    uint32_t seq;            /* 0 = entrada libre */
    int32_t  status;         /* 0 o código de error */
    uint32_t out_cap, out_len;
    uint8_t  dest;           /* nodo al que se le pidió */
    uint8_t  reserved[7];
} Pending;
static_assert(sizeof(Pending) == 64, "Pending");
```

`Config` agrupa `uint64 dsm_base`, cuatro `uint32` (`page_size`, `pages_per_node`, `heartbeat_ms`, `timeout_ms`), `uint16 port_base`, `uint16 num_hosts`, 4 bytes reservados y `char hosts[32][64]`, con `static_assert(sizeof(Config) == 2080)`. La tabla de páginas es un arreglo de `N * pages_per_node` entradas de 16 B (128 KB con 32 nodos y 256 páginas por nodo), la de nodos, un arreglo de `MAX_NODES` entradas `NodeInfo` más un arreglo paralelo de `uint64` con el último instante en que se supo de cada nodo.

5. Protocolo de mensajes

5.1 Encabezado

Todos los mensajes empiezan con el mismo encabezado de 16 bytes, seguido de `payload_len` bytes de carga. Los enteros de más de un byte viajan en orden de red: las estructuras se convierten campo por campo al enviar y al recibir. TCP es un flujo de

bytes, así que el receptor lee primero 16 bytes y luego la carga. Una carga mayor que `page_size` es un error y se cierra la conexión.

Despl.	Campo	Tipo	Tamaño	Significado
0	`type`	`uint8`	1	Tipo de mensaje (sección 5.2)
1	`sender`	`uint8`	1	`node_id` del emisor
2	`status`	`uint8`	1	0 si todo está bien o el valor absoluto del código de error; 0 en las solicitudes
3	`flags`	`uint8`	1	En `PAGE_REQUEST` y `PAGE_DATA`: 0 = S, 1 = M
4	`seq`	`uint32`	4	Número de secuencia: la respuesta repite el de la solicitud
8	`page`	`uint32`	4	Número de página, o `PAGE_NONE` si no aplica
12	`payload_len`	`uint32`	4	Bytes de carga que siguen

5.2 Mensajes

Los mensajes viajan por tres canales: **datos** (rol 0), **control** (`ctrl`., rol 1) y **heartbeat** (`hb`, rol 2), de modo que una transferencia de página no retrase un `HEARTBEAT`. Las respuestas con error llevan `status` distinto de cero y no llevan carga. Los códigos de error son los de la sección 6.

Tipo	Canal	Emisor y receptor	Campos y carga	Respuesta	Errores
`JOIN` (1)	ctrl.	nodo nuevo a los demás	Carga: `NodeInfo` propio	`JOIN_ACK`	(los reporta `JOIN_ACK`)
`JOIN_ACK` (2)	ctrl.	nodo a nodo nuevo	Carga: `NodeInfo` propio	(ninguna)	`INVAL` (datos del `JOIN` no coinciden con la configuración)
`HEARTBEAT` (3)	hb	cada nodo a los demás	Sin carga	(ninguna)	
`PAGE_REQUEST` (4)	datos	solicitante a home, y home a dueño	`page`; `flags` = permiso pedido (S o M)	`PAGE_DATA`	`BADADDR` (página fuera de rango o libre), `NODE_DOWN`
`PAGE_DATA` (5)	datos	home a solicitante, y	`page`; `flags` = permiso concedido;	(ninguna)	los de `PAGE_REQUES`

Tipo	Canal	Emisor y receptor	Campos y carga	Respuesta	Errores
		dueño a home	carga: `page_size` bytes, o ninguna si el solicitante ya tiene copia		`T`
`INVALIDATE` (6)	datos	home a cada copia	`page`	`INVALIDATE_ACK`	`NODE_DOWN` (se omite el nodo)
`INVALIDATE_ACK` (7)	datos	copia a home	`page`	(ninguna)	
`ALLOC_REQUEST` (8)	ctrl.	solicitante a un home	Carga: `AllocReq`	`ALLOC_REPLY`	`NOMEM`, `INVAL`
`ALLOC_REPLY` (9)	ctrl.	home a solicitante	`page` = primera página, o `PAGE_NONE`	(ninguna)	los de `ALLOC_REQUEST` y `FREE`
`FREE` (10)	ctrl.	solicitante a home	`page` = primera página de la asignación	`ALLOC_REPLY`	`BADADDR`
`QUERY_REQUEST` (11)	ctrl.	solicitante a un nodo	Carga: `QueryReq` (`kind` 0 = estadísticas, 1 = tabla de páginas)	`QUERY_REPLY`	`INVAL`
`QUERY_REPLY` (12)	ctrl.	nodo a solicitante	Carga: un `NodeStats`, o hasta `QUERY_MAX` `PageEntry`	(ninguna)	

En `PAGE_REQUEST` el receptor decide el papel por su posición. Una consulta de páginas devuelve como máximo `QUERY_MAX` entradas (4096 B); para rangos mayores se repite con otro `first_page`.

5.3 Atención de una solicitud en el home

El hilo de servicio del home toma `home_lock` de la página y hace lo siguiente. Cuando el home es también el solicitante, el dueño o una copia, esa parte se hace con una llamada local en lugar de un mensaje.

1. Si la página está libre, responde `PAGE_DATA` con `BADADDR`.
2. Si el solicitante ya está en el conjunto de copias, no hace falta enviar la página: responde sin carga. En caso contrario envía `PAGE_REQUEST` al dueño y espera su `PAGE_DATA`. Si la solicitud es de lectura, el dueño pasa a S y conserva su copia; si es de escritura, el dueño pasa a I.
3. Si la solicitud es de escritura, envía `INVALIDATE` a las demás copias (las que no son el solicitante ni el dueño) y espera todos los `INVALIDATE_ACK`. Cada copia pasa a I.

4. Actualiza la entrada: en una lectura agrega al solicitante al conjunto de copias; en una escritura el solicitante es el nuevo dueño y el único en el conjunto.
5. Envía `PAGE_DATA` al solicitante con el permiso concedido y libera `home_lock`.

Como todo mensaje que cambia el estado de la página `p` en cualquier nodo sale de su home, y los mensajes de un nodo a otro mantienen su orden en una conexión TCP, un nodo siempre ve los cambios de `p` en el orden en que el home los decidió.

`ALLOC_REQUEST` recorre la tabla del home buscando el primer grupo de `npages` páginas libres consecutivas, si lo encuentra, escribe en cada una el `proc_id` y el orden, la deja en M y llena de ceros en el home, y responde la primera página. `FREE` invalida las copias de cada página de la asignación como en el paso 3 y la deja libre y en I.

6. Borrador de la API

La API se declara en `dsm.h` con `extern "C"`. Todas las funciones devuelven 0 o un valor positivo si tienen éxito, y un código de error negativo si fallan; `dsm_strerror` lo convierte en texto. Los tipos públicos son `dsm_node_t` (`NodeInfo`), `dsm_page_t` (`PageEntry`) y `dsm_stats_t` (`NodeStats`), con los campos en el orden de bytes del equipo.

```
typedef enum {
    DSM_OK = 0,        DSM_E_INVAL = -1,  DSM_E_CONFIG = -2,    DSM_E_NOMEM = -3,
    DSM_E_BADADDR = -4, DSM_E_NODE_DOWN = -5, DSM_E_TIMEOUT = -6, DSM_E_NET = -7
} dsm_err_t;

int         dsm_init(const char *config_path, int node_id);
void        dsm_shutdown(void);
int         dsm_alloc(size_t size, uint16_t proc, void **ptr);
int         dsm_free(void *ptr);
int         dsm_nodes(dsm_node_t *out, int max);
int         dsm_stats(int node_id, dsm_stats_t *out);
int         dsm_page_table(int node_id, uint32_t first_page, uint32_t count,
                           dsm_page_t *out);
size_t      dsm_page_size(void);
const char *dsm_strerror(int err);
```

Función	Parámetros	Retorno	Errores
`dsm_init`	`config_path`: ruta de `dsm.conf`; `node_id`: posición de este nodo en `host`	0	`INVAL` (`node_id` fuera de la lista), `CONFIG` (archivo inválido o `mmap` falla), `NET` (no puede abrir los puertos)
`dsm_shutdown`	ninguno	nada	ninguno
`dsm_alloc`	`size`: bytes pedidos, redondeados a páginas; `proc`: número de proceso de la aplicación, de 1 a 65535; `ptr`: recibe la dirección	0	`INVAL` (`size` 0 o mayor que `pages_per_node` páginas), `NOMEM` (ningún home tiene espacio), `NODE_DOWN`, `TIMEOUT`

Función	Parámetros	Retorno	Errores
`dsm_free`	`ptr`: dirección devuelta por `dsm_alloc`	0	`BADADDR` (no es el inicio de una asignación)
`dsm_nodes`	`out`: arreglo de `NodeInfo`; `max`: su capacidad	cantidad de nodos copiados	`INVAL`
`dsm_stats`	`node_id`: nodo consultado; `out`: estadísticas	0	`INVAL`, `NODE_DOWN`, `TIMEOUT`
`dsm_page_table`	`node_id`: nodo cuyo rango se consulta; `first_page` y `count`: páginas pedidas (hasta 256); `out`: arreglo de `PageEntry`	entradas copiadas	`INVAL`, `NODE_DOWN`, `TIMEOUT`
`dsm_page_size`	ninguno	tamaño de página en bytes	ninguno
`dsm_strerror`	`err`: código de error	texto fijo del error	ninguno

`dsm_alloc` prueba primero el rango del nodo local y luego el de los demás nodos en línea, en orden de `node_id`, enviando `ALLOC_REQUEST` a cada uno hasta que uno responda con éxito. El `proc_id` guardado en la tabla de páginas es `(node_id << 16) | proc`, lo que identifica sin coordinación al proceso y al nodo donde corre. Las estadísticas de todo el sistema se obtienen llamando a `dsm_stats` para cada nodo de `dsm_nodes`.

Bibliografía

[1] E. Arias Méndez, "Proyecto Programado: Implementación de un Sistema de Memoria Compartida Distribuida (DSM) con Paginación bajo Demanda y Coherencia MSI," IC-6600 Principios de Sistemas Operativos, Escuela de Ingeniería en Computación, Instituto Tecnológico de Costa Rica, Cartago, Costa Rica, Oct. 2026.

[2] K. Li and P. Hudak, "Memory coherence in shared virtual memory systems," *ACM Trans. Comput. Syst.*, vol. 7, no. 4, pp. 321-359, Nov. 1989.

[3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 9th ed. Hoboken, NJ, USA: John Wiley & Sons, 2013.

[4] M. Kerrisk, "The Linux man-pages project: mmap(2), mprotect(2), memfd_create(2), sigaction(2)," man7.org. [Online]. Available: https://man7.org/linux/man-pages/. Accessed: Nov. 2, 2026.